Fine-Tuning & Model Training

Intensive self-study curriculum for getting hands-on and proficient

Recipient: Ian
Date: June 2026
Format: work through it module by module, hands-on where indicated

---

1. What this is and how to use it

This is a structured learning path to make you genuinely comfortable with model training and fine-tuning, end to end. The point is depth, not speed. By the time you finish I want you to actually understand how task-specific fine-tuning works, to have run several real trainings yourself, and to be able to reason about how a generic, reusable training pipeline would be designed.

Each module below follows the same shape: a short orientation on why the topic matters, a list of concrete questions you should be able to answer afterwards, the best current resources to learn from, and a hands-on step where it applies. Don't just read. The hands-on runs are where the understanding actually forms. Reading about LoRA for an hour teaches less than running one 20-minute LoRA job and watching the loss curve.

Take a full first day to go broad across the foundations and run your first training, then one or two more days to go deeper into embeddings, multimodal, and the evaluation and merging pieces. No deliverable pressure during the learning; there is a short write-up at the end so we both know which areas you're solid on.

2. The two directions this covers

The curriculum spans two technical directions, because the work ahead can pull on either. Keep both in mind as you go, since they exercise slightly different parts of the topic. The common core is the same for both, so the learning transfers regardless of which one you end up applying first.

Direction A — task-specific text fine-tuning

Training small task-specific LoRA specialists on an open instruct model (think 8B-class models), each one focused on a single well-defined task and trained with supervised fine-tuning on task examples. Alongside that, fine-tuning an embedding model on a domain corpus and merging it back with the base model to gain domain accuracy while keeping general capability. Both lean on synthetic data generation to expand a small training set, a held-out gold set to measure whether the model actually learned, and drift checks so it doesn’t get worse at general tasks. This is the direction to assume you’ll apply first.

Direction B — multimodal vision-language fine-tuning

Working with a vision-encoder-plus-LLM model: an image plus a question goes in, and a structured analysis comes out (for example, inspecting a scene and identifying what is present, missing, or out of place). That means building an annotated image-text dataset and

fine-tuning the model for a specific visual domain. It is a realistic possibility you may need this on short notice, so one module below is dedicated to it.

The common core across both: how training data must be shaped, how a fine-tuning run actually works, what LoRA and QLoRA are, how to evaluate honestly, and which knobs matter. Master that core and both directions become approachable.

3. Suggested order

Work through the modules roughly in order, since later ones assume the earlier ones. Don’t spend the whole first stretch only reading. Get through the foundations (Modules A–C) and then move straight into your first real training in Module E before going further. Getting one LoRA/QLoRA run completed early, with a loss curve you can read and a before/after test, is what makes the rest click.

After that first run, continue in sequence: synthetic data and evaluation (D, F), embedding fine-tuning and merging (G, H), the multimodal module with its own small run (I), then commercial APIs and serving (J, K), and finally the short write-up in Section 7. Each module tells you when there’s a hands-on step; do it before moving on rather than batching all the reading first.

4. Learning modules

Module A — Foundations: when to fine-tune, and how

Before any technique, get the mental model straight. Fine-tuning changes a model's weights so it gets better at a specific task or domain. Retrieval (RAG) does not change weights; it just feeds relevant context at inference time. They solve different problems and are often combined. A common myth is that fine-tuning can't teach new behavior or that RAG always beats it; that's wrong. RAG augments what the model sees; fine-tuning changes what the model is.

Questions you should be able to answer

- What is the difference between pretraining, fine-tuning, and RAG, and when do you reach for each?
- What does “task-specific” or “instruction” fine-tuning mean, versus continued pretraining?
- What is full fine-tuning versus parameter-efficient fine-tuning (PEFT), and why is full fine-tuning usually unnecessary?
- What is supervised fine-tuning (SFT), and how does it differ from preference methods like DPO or RL approaches like GRPO?

Resources

Unsloth — Fine-tuning LLMs Guide (start here, it frames the whole field cleanly): unsloth.ai/docs/get-started/fine-tuning-llms-guide

Hugging Face — TRL / SFT documentation for the SFT concept and trainer: huggingface.co/docs/trl

Module B — LoRA and QLoRA

This is the core technique you'll use most. Instead of updating all of a model's billions of weights, LoRA freezes the original weights and adds small low-rank adapter matrices (A and B) to selected layers, training only those. That's a tiny fraction of the parameters, which is why it fits on modest GPUs. QLoRA goes further: it quantizes the frozen base model to 4-bit so even large models fit in little memory, while still training the adapters. Understand rank (r), alpha, which modules you target (attention projections, MLP), dropout, and what “merging” an adapter back into the base model means.

Questions you should be able to answer

- What are the LoRA A and B matrices, and what does the rank r control? What does alpha do?
- Which layers/modules do you typically attach LoRA to, and why might you attach to more or fewer?
- What exactly does QLoRA quantize, and what is NF4? What is the memory-versus-quality trade-off?
- When do you keep the adapter separate (for multi-adapter serving) versus merge it into the base weights?
- Roughly how much VRAM does a LoRA versus QLoRA run of an 8B model need?

Resources

Hugging Face PEFT docs (LoRA/QLoRA, the canonical library): huggingface.co/docs/peft

Unsloth fine-tuning guide section on LoRA vs QLoRA (clear, practical): unsloth.ai/docs/get-started/fine-tuning-llms-guide

Module C — Training datasets and data formats

Model quality is mostly dataset quality. For supervised fine-tuning you need examples in a consistent instruction or chat format (usually JSONL), each with the input the model will see and the output you want it to produce. Understand chat templates (how messages get formatted for a given model), the train/validation/eval split, and why a held-out evaluation set that the model never trains on is non-negotiable: it's the only way to know whether the model learned the task or just memorized the training data.

Questions you should be able to answer

- What does an SFT example look like in practice (system/user/assistant or prompt/completion)? What is JSONL?
- What is a chat template and why must it match the model you're training?
- Why split into training, validation, and a held-out eval/gold set? What goes in each, and what must never leak between them?
- How many examples are realistic for a task-specific fine-tune, and how does data quality trade against quantity?
- Why keep a held-out slice of real data as the gold set rather than commissioning a separate labeled set?

Resources

Hugging Face — dataset formats for TRL/SFT: huggingface.co/docs/trl/dataset_formats

Hugging Face Datasets library docs (loading, splitting, mapping): huggingface.co/docs/datasets

Module D — Synthetic data generation and quality filtering

Real labeled data is often too small. Synthetic data generation (SDG) uses an LLM to expand it. The key discipline is that SDG should expand the variety of phrasings, not invent new facts: take a real example, generate several paraphrases of the input while keeping the verified output, so the model sees many ways of expressing the same thing. Then an LLM-as-judge step filters out weak generations. This is how a few thousand real examples become a more robust training set without fabricating knowledge.

Questions you should be able to answer

- What is the difference between expanding phrasing variety and fabricating new knowledge, and why does the distinction matter for correctness?
- How does an LLM-as-judge quality filter work, and what failure modes does it catch?
- How do you keep synthetic data balanced (length, type, persona) instead of skewed?
- Why would we run SDG with a local model rather than a cloud API in a data-protection-sensitive setting?

Resources

Distilabel docs (the SDG framework we reference; pipelines, judges, personas): distilabel.argilla.io

Module E — Running a training: mechanics and pitfalls (hands-on)

Now actually run one. Understand the core hyperparameters and what they do: learning rate, number of epochs, batch size, gradient accumulation (simulating a bigger batch on a small GPU), gradient checkpointing (trading compute for memory), and precision (bf16). Learn to read a loss curve, to spot overfitting (training loss drops, validation loss rises), and to recognize catastrophic forgetting (the model gets better at your task but worse at everything else). The standard mitigation is mixing a small share of general data back into the training set.

Questions you should be able to answer

- What do learning rate, epochs, batch size, and gradient accumulation each control, and what happens if each is too high or too low?
- How do you read a training/validation loss curve, and what does overfitting look like?
- What is catastrophic forgetting, how do you detect it, and how does mixing in general data help?
- What is gradient checkpointing and when do you need it?

Hands-on (your first real training)

Open an Unsloth Colab notebook for a small Qwen3 model, run it end to end on a free GPU, then swap in a small custom dataset of your own. Start from the notebook list at github.com/unslothai/unsloth and the Qwen guide at unsloth.ai/docs. Goal: one completed

run, a loss curve you can interpret, and a before/after comparison of the model's output on a couple of prompts.

Module F — Evaluation and drift

A training is only meaningful if you can measure whether it worked. Two axes: did the model get better at the target task (measured on the held-out gold set), and did it get worse at general capability (measured on public benchmark sentinels). A sound practice is to set explicit acceptance thresholds in advance, for example a minimum domain-accuracy gain and a maximum tolerated general-capability regression, and to report honestly when they aren't met rather than overselling.

Questions you should be able to answer

- How do you build and use a held-out gold set to measure task learning?
- What is a drift sentinel (e.g. an MMLU subset for LLMs, an MTEB subset for embeddings) and how does it guard against regression?
- Why set acceptance criteria before training rather than after?
- What does it mean to fall back to pure RAG if a fine-tune doesn't clear the bar?

Resources

MTEB (Massive Text Embedding Benchmark) for retrieval/embedding evaluation: github.com/embeddings-benchmark/mteb

Hugging Face — lighteval / evaluation tooling for LLM benchmarks: github.com/huggingface/lighteval

Module G — Embedding model fine-tuning (hands-on)

Different objective from LLM fine-tuning. Embedding models map text into vectors so that similar texts sit close together; you fine-tune them with contrastive learning on pairs (a query and its correct document as a positive, plus negatives). The hard part is hard-negative mining: finding wrong documents that look superficially similar, because those teach the model the most. Understand the common loss (MultipleNegativesRankingLoss), Matryoshka training (so one model gives usable shorter vectors too), and retrieval metrics (NDCG@k, recall@k, MRR). This is the embedding fine-tuning workflow.

Questions you should be able to answer

- What is contrastive learning, and what are positives and negatives in an embedding training pair?
- What is hard-negative mining, why does it matter, and what's the risk of mining false negatives?
- What does MultipleNegativesRankingLoss do? What is Matryoshka representation training?
- What do NDCG@k, recall@k, and MRR measure, and how do you compute a before/after on your own data?

Resources

Sentence Transformers — training overview, losses, and hard-negative mining (the primary library): sbert.net/docs/sentence_transformer/training_overview.html

Unsloth — embedding fine-tuning notebooks (incl. Qwen3-Embedding and EmbeddingGemma, QLoRA-friendly): unsloth.ai/docs/basics/embedding-finetuning

Hands-on

Fine-tune a small embedding model on a small domain dataset, mine hard negatives, and measure a retrieval metric before and after. Even a tiny improvement that you measured yourself teaches the whole loop.

Module H — Model merging

After fine-tuning an embedding model on a narrow domain, merging it back with the base model recovers general capability while keeping the domain gain. Understand weight-merging methods (TIES, SLERP), the merge ratio (alpha) and why you sweep it across a few values to find the sweet spot, and why merging is effectively free compared to retraining.

Questions you should be able to answer

- Why merge a fine-tuned model with its base at all? What problem does it solve?
- What's the difference between TIES and SLERP at a conceptual level?
- What does the alpha ratio control, and why sweep it (e.g. 0.3 to 0.7)?

Resources

MergeKit (the standard merging toolkit; read the README and method list): github.com/arcee-ai/mergekit

Module I — Multimodal / vision-language fine-tuning (hands-on)

This is the module for the multimodal context. A vision-language model (VLM) has three parts: a vision encoder that turns an image into visual tokens, a merger/projector that aligns those with the language model's space, and the LLM itself. The standard, memory-friendly recipe for domain adaptation is to freeze the vision encoder and the merger and train only the LLM with LoRA. Data is image-plus-text: each example pairs an image with a prompt and the desired answer, formatted through the model's multimodal chat template. This maps directly onto visual-inspection use cases generally (image in, structured assessment out).

Questions you should be able to answer

- What are the three blocks of a VLM, and which do you typically freeze versus train for domain adaptation, and why?
- How does a training example look when it includes an image? How does the chat template handle the image?
- Why is image resolution / number of visual tokens a memory and data-design consideration?
- When would you also unfreeze the vision encoder, and what's the risk?

Resources

Hugging Face Cookbook — fine-tuning a VLM (Qwen2-VL) with TRL, free and end-to-end: huggingface.co/learn/cookbook/en/fine_tuning_vlm_trl

Merve Noyan's smol-vision repository (a collection of practical vision/multimodal notebooks): github.com/merveenoyan/smol-vision

Unsloth vision fine-tuning (select which parts of the VLM to train; free Colab notebooks): unsloth.ai/docs

Hands-on

Run a small VLM fine-tune (for example Qwen2-VL or Qwen2.5-VL on a small image-QA dataset such as ChartQA from the cookbook). You don't need a domain dataset yet; the goal is to understand the multimodal data flow and the freeze-versus-train decision concretely.

Module J — Commercial fine-tuning APIs

Besides local/open-source training, managed cloud fine-tuning exists: you upload a dataset, the provider trains and hosts the model, and you call it by API. Know this path because it's sometimes the right trade-off (no infrastructure, fast to try), and because comparing local open-source training against commercial API models is a common evaluation question. Two important current realities: Google Vertex AI offers supervised tuning for Gemini including image tuning, which is directly relevant to a multimodal use case; OpenAI is winding down its fine-tuning platform and is closed to new users, so don't plan around it. Anthropic has no native fine-tuning (Claude tuning is only via Amazon Bedrock for specific models).

Questions you should be able to answer

- What's the workflow for a managed fine-tune (data format, upload, job, hosted endpoint)?
- When is a managed API the right call, and when is local open-source training better (cost, data protection, control)?
- When would a data-protection requirement push toward local training rather than cloud APIs?

Resources

Google Vertex AI — Gemini supervised tuning, including image tuning (the live, multimodal-capable path): docs.cloud.google.com/vertex-ai/generative-ai/docs/models/tune_gemini/image_tune

Google Codelab — hands-on Gemini supervised fine-tuning on Vertex AI end to end: codelabs.developers.google.com/codelabs/production-ready-ai-with-gc/9-ai-finetuning/finetune-gemini-vertex-ai

Module K — Serving fine-tuned models

Lighter touch, but know how a trained model gets used in production. Understand serving a model with an inference engine (vLLM), and especially multi-adapter serving: many LoRA adapters running on a single base model, switched per request, which is an efficient way to serve several task specialists cheaply from one container. Also know how to export to formats like GGUF for local runtimes (Ollama, llama.cpp).

Questions you should be able to answer

- What does an inference engine like vLLM do, and why use it over plain transformers for serving?
- How does multi-LoRA serving work, and why is it efficient for hosting several specialists?

- What is GGUF and when would you export to it?

Resources

vLLM docs, including LoRA adapter serving: docs.vllm.ai

5. Free GPU environments to practice on

You don't need our servers for any of the learning. Use the free notebook environments from the GPU resource document I'm sharing alongside this. For the hands-on runs, prioritize these three, in this order:

- **Kaggle Notebooks** and **Google Colab** — fastest start, free GPU, and almost every notebook linked above is built for them. Use these for the LoRA, QLoRA, embedding, and small VLM runs.
- **Amazon SageMaker Studio Lab** — free, separate from a normal AWS account, GPU runtime in 4-hour sessions. Good clean second environment. Follow the safety notes in the GPU document: only via the Studio Lab URL, never enter AWS keys or billing, never upload sensitive data.

The other services in that document (Lightning AI, Hugging Face ZeroGPU/PRO, Modal) are useful as backups or for specific cases, but Kaggle and Colab will carry most of the hands-on. Keep your runs small; the goal is understanding, not big models. Save your notebook and any results out of the session, since free runtimes are time-limited and reset.

6. Concrete hands-on milestones

These are the runs that actually make you proficient. Aim to complete all of them as you work through the modules. Small models and small datasets throughout.

1. **LoRA/QLoRA SFT:** fine-tune a small Qwen3 model on a small instruction dataset with Unsloth, end to end. Interpret the loss curve, test before/after.
2. **Dataset from scratch:** hand-build a tiny task-specific dataset in proper instruction/JSONL format and train on it, so the data-shaping step is concrete, not abstract.
3. **Synthetic data:** use an LLM to paraphrase a handful of examples into a larger set (variety, not new facts), then eyeball or judge-filter the quality.
4. **Embedding fine-tune:** fine-tune a small embedding model with hard-negative mining; measure a retrieval metric before and after on your own small set.
5. **Model merge:** merge a fine-tuned model with its base using MergeKit and observe the effect.
6. **Multimodal fine-tune:** run a small VLM fine-tune on an image-QA dataset to understand the multimodal flow and the freeze-versus-train choice.
7. **Managed path (optional but valuable):** run one Vertex AI Gemini supervised tune via the codelab to feel the managed workflow, contrasted with local training.

7. Short write-up at the end

When you've worked through it, send me a brief note, nothing heavy. For each hands-on milestone: what you ran, which environment, the VRAM and time it took, and one or two sentences on what you learned or found surprising. Then a short paragraph on how you'd approach designing a generic, reusable training pipeline now that you understand the pieces: how curated source data would become a training set, how you'd run the LoRA and the embedding fine-tune, and how you'd evaluate honestly. That tells us both where you're solid and where to go deeper next.

No rush on any of this. The aim is real, durable understanding you can build on, not a fast result. Ask along the way if anything is unclear, and lean on AI assistants (Claude included) to explain concepts, debug notebooks, and suggest further examples as you go.